

**Методичні вказівки до виконання лабораторних робіт
з дисципліни: Поглиблене вивчення JavaScript
Лабораторна робота №1
Розробка базового застосунку керування задачами (To-Do List)**

1.1 Мета роботи

Ознайомлення з основними принципами роботи фреймворку Vue 3, формування практичних навичок створення односторінкових застосунків, засвоєння механізмів реактивності, директив, обробки подій та компонентного підходу.

1.2 Загальні відомості

Сучасні вебзастосунки орієнтовані на створення динамічних інтерфейсів, які взаємодіють із користувачем без перезавантаження сторінки. Такі застосунки називаються односторінковими (Single Page Application, SPA).

Фреймворк Vue.js є одним із найпоширеніших інструментів для створення SPA. Його основною перевагою є реактивна модель даних, яка забезпечує автоматичне оновлення інтерфейсу при зміні стану застосунку.

У межах даної лабораторної роботи розглядаються базові можливості Vue 3, що є основою для подальшого вивчення технології.

1.3 Теоретичні відомості

Vue.js використовує компонентний підхід до побудови інтерфейсу. Компонент є самостійною частиною застосунку, яка містить розмітку, логіку та стилі. Такий підхід дозволяє створювати масштабовані та зручні для підтримки застосунки.

Ключовою концепцією Vue є реактивність. Реактивні змінні дозволяють автоматично відслідковувати зміни даних і оновлювати відповідні елементи інтерфейсу. Це значно спрощує розробку порівняно з традиційною роботою з DOM.

Для взаємодії між даними та шаблоном використовуються директиви. Вони дозволяють виконувати різні операції безпосередньо у

розмітці, зокрема організовувати умовне відображення, ітерацію по колекціях та двосторонній зв'язок даних.

Обробка подій реалізується шляхом прив'язки дій користувача до відповідних методів. Це забезпечує інтерактивність застосунку та можливість реагування на дії користувача.

1.4 Інструкція зі встановлення та налаштування середовища

Для виконання лабораторної роботи необхідно підготувати середовище розробки.

На першому етапі встановлюється платформа Node.js, яка використовується для запуску JavaScript поза браузером та керування залежностями проєкту. Рекомендується використовувати актуальну LTS-версію. Після встановлення необхідно перевірити її працездатність за допомогою команд:

```
node -v  
npm -v
```

A terminal window with a dark background. The first command is 'node -v' and the output is 'v23.10.0'. The second command is 'npm -v' and the output is '10.9.2'. The prompt is '~ %' for both.

```
~ % node -v  
v23.10.0  
~ % npm -v  
10.9.2
```

Наступним кроком є створення нового проєкту із використанням інструмента Vite. Для цього у командному рядку виконується команда:

```
npm create vite@latest
```

```
~ % npm create vite@latest
Need to install the following packages:
create-vite@9.0.3
Ok to proceed? (y) y

> npx
> create-vite

|
|   Project name:
|   to-do list
|
|   Package name:
|   to-do-list
|
|   Select a framework:
|   Vue
|
|   Select a variant:
|   JavaScript
|
|   Install with npm and start now?
|   Yes
|
|   Scaffolding project in /Users/[redacted]/to-do list...
|
|   Installing dependencies with npm...
|
added 36 packages, and audited 37 packages in 9s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
|
|   Starting dev server...
|
> to-do-list@0.0.0 dev
> vite

VITE v8.0.3 ready in 476 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

У процесі створення проєкту необхідно обрати шаблон Vue та задати назву проєкту.

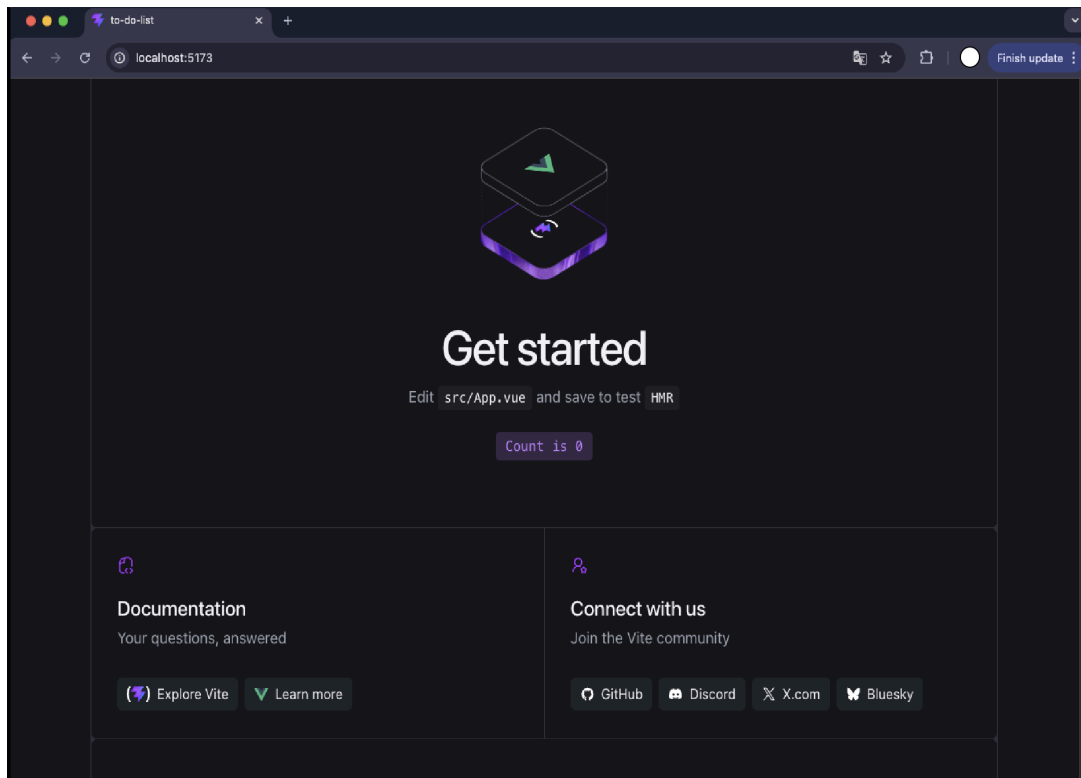
Після цього необхідно перейти до каталогу проєкту та встановити залежності:

```
cd <назва_проєкту>
npm install
```

Запуск застосунку здійснюється командою:

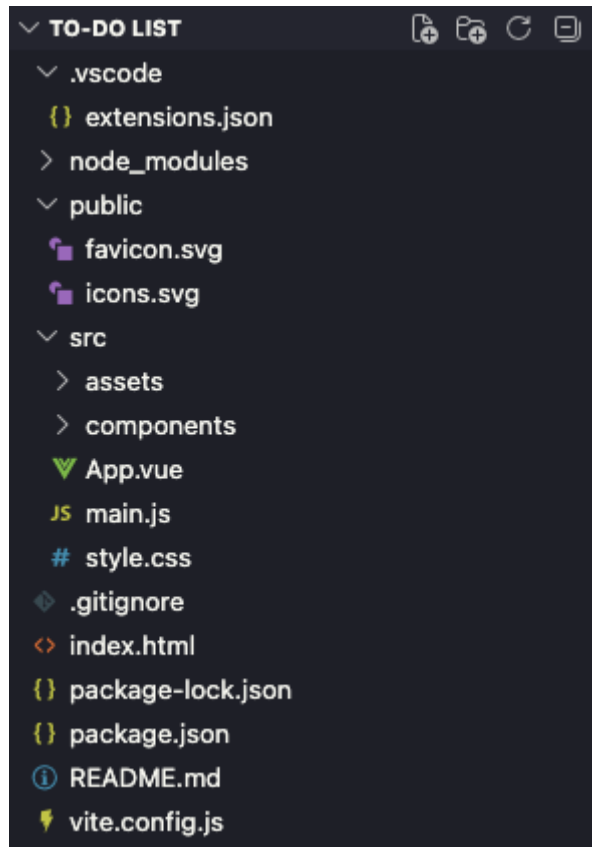
```
npm run dev
```

Після запуску у консолі відображається локальна адреса, за якою застосунок доступний у браузері.



1.5 Структура проєкту

Після створення формується стандартна структура Vue-проєкту.



Основними елементами є:

- index.html - головний HTML-файл, що містить точку підключення застосунку;
- main.js - файл ініціалізації застосунку;
- App.vue - кореневий компонент;
- components/ - каталог для користувацьких компонентів;
- assets/ - каталог для статичних ресурсів.

Файл main.js відповідає за створення екземпляра застосунку та його монтування у DOM. Кореневий компонент App.vue є основою інтерфейсу.

У межах лабораторної роботи рекомендується створити окремі компоненти для відображення списку задач та окремих елементів списку, що відповідає принципам компонентного підходу.

1.6 Порядок виконання роботи

Виконання лабораторної роботи здійснюється у такій послідовності:

1. Встановити середовище розробки (Node.js) та перевірити його працездатність.

2. Створити новий проєкт із використанням інструмента Vite, обравши шаблон Vue 3.
3. Встановити залежності проєкту та запустити його у режимі розробки.
4. Підготувати базову структуру застосунку:
 - a. очистити стандартний шаблон;
 - b. залишити мінімальну розмітку у кореновому компоненті.
5. Створити у кореновому компоненті змінні стану для:
 - a. збереження списку задач;
 - b. збереження значення поля введення;
 - c. збереження поточного режиму фільтрації.
6. Реалізувати форму введення нової задачі:
 - a. створити поле введення;
 - b. організувати двосторонній зв'язок даних;
 - c. додати кнопку підтвердження.
7. Реалізувати функціонал додавання задачі:
 - a. перевіряти коректність введених даних;
 - b. додавати нову задачу до списку;
 - c. очищати поле введення після додавання.
8. Організувати відображення списку задач:
 - a. використати ітерацію по колекції;
 - b. забезпечити унікальність елементів списку.
9. Реалізувати зміну стану задачі:
 - a. додати можливість позначення задачі як виконаної;
 - b. забезпечити відповідне відображення у інтерфейсі.
10. Реалізувати видалення задачі:
 - a. додати елемент керування для видалення;
 - b. забезпечити оновлення списку після видалення.
11. Реалізувати фільтрацію задач:
 - a. передбачити режими відображення (усі, активні, виконані);
 - b. організувати відображення відповідно до обраного режиму.
12. (Додатково) Виконати розподіл застосунку на компоненти:
 - a. виділити компонент списку задач;
 - b. виділити компонент окремої задачі.
13. Перевірити працездатність застосунку:
 - a. коректність додавання задач;
 - b. коректність зміни стану;

- c. роботу фільтрації;
- d. відсутність помилок під час виконання.

1.7 Приклад реалізації застосунку

Реалізація застосунку базується на використанні реактивних змінних та обробки подій.

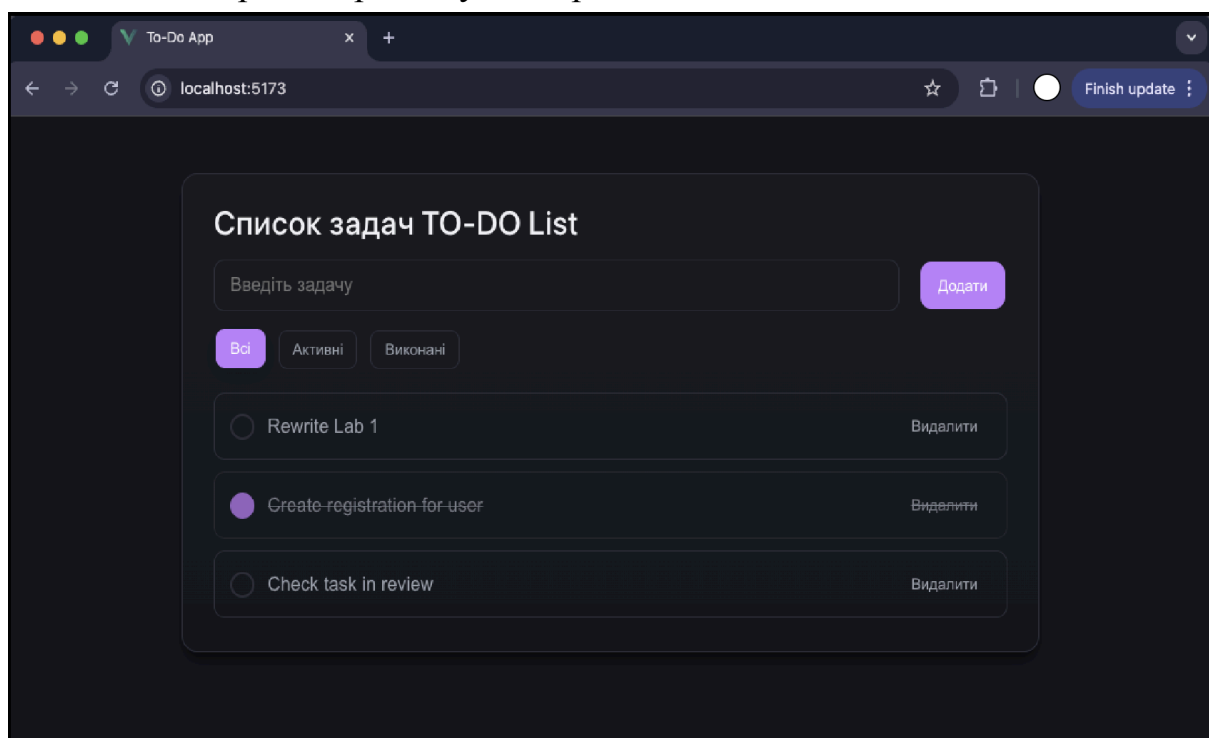
Список задач представляється у вигляді масиву об'єктів, кожен із яких містить текст задачі, ідентифікатор та статус виконання.

Додавання задачі здійснюється шляхом обробки відповідної події. Після додавання список автоматично оновлюється.

Відображення задач реалізується через ітерацію по колекції. Для кожного елемента передбачено можливість зміни стану та видалення.

Зміна стану задачі реалізується шляхом зміни відповідного поля об'єкта. Відображення статусу здійснюється із використанням умовного форматування.

Фільтрація задач реалізується шляхом формування похідного списку залежно від обраного режиму відображення.



1.8 Завдання до виконання

У процесі виконання лабораторної роботи необхідно:

- створити проєкт із використанням Vue 3;
- реалізувати введення нових задач;
- забезпечити додавання задач до списку;
- організувати відображення списку задач;
- реалізувати зміну стану задач;
- реалізувати видалення задач;
- додати фільтрацію задач;
- за можливості реалізувати компонентну структуру застосунку.

1.9 Вимоги до захисту

Усі лабораторні роботи виконуються у вигляді окремих проєктів або модулів одного проєкту. Код має бути структурованим, із чітким поділом на компоненти та дотриманням принципів чистого коду.

Результати роботи студент повинен представити у вигляді:

- **вихідний код (репозиторій);**
- короткого звіту з описом мети, теоретичних відомостей та реалізації;
- демонстрації працездатності застосунку.

Під час захисту особлива увага приділяється розумінню використаних технологій, логіці побудови застосунку та вмінню пояснити власний код.

1.10 Оцінювання лабораторних робіт

Кожна лабораторна робота оцінюється максимально у **25 балів**. Оцінка залежить від повноти виконання завдання, працездатності застосунку та здатності студента пояснити свою реалізацію під час захисту.

Чим повніше реалізовано функціонал і стабільніше працює додаток, тим вищий бал. Частково виконані або некоректні роботи оцінюються нижче.

1.11 Контрольні питання

1. Що таке SPA?
2. Основні принципи роботи Vue.js.
3. Що таке реактивність?
4. Яке призначення директив?
5. Як реалізується обробка подій?
6. Що таке компонент?